
Clock Divider and Counter Design

on a Zynq-7020 FPGA (xc7z020clg484-2)

Name: Marium Waleed

Date: July 2026

1 Introduction

This report covers a simple FPGA design built on the Zynq-7020 (xc7z020clg484-2). A PLL generates a clock from the board input clock. That clock is divided down to a very slow clock, which drives a 3-bit counter. The design was simulated, synthesized, and implemented.

The design has four parts:

- A PLL IP core (`clk_wiz_0`) that outputs 8 MHz and a locked signal.
- A `clk_divider` module that divides the PLL clock down further.
- A counter module that counts on every rising edge of the slow clock.
- A `top` module connecting all of the above.

Note: On hardware, the divider targets 1 Hz. For simulation, a faster divider value was used so the counter could be observed toggling in a reasonable simulation time, giving an effective clock of 1 kHz instead of 1 Hz.

2 Source Code

2.1 Top Module

The top module connects the PLL, the clock divider, and the counter.

```
module top(  
    input clk,  
    input rst_n,  
    output [2:0] counter  
);  
  
wire clk_counter;    // 8 MHz from PLL  
wire lock_counter;    // PLL locked  
wire clk_lhz;        // 1 Hz from divider  
  
counter u_count(  
    .clk(clk_lhz),  
    .rst_n(lock_counter),  
    .counter(counter)  
);  
  
clk_divider u_div(  
    .clk(clk_counter),  
    .rst_n(lock_counter),  
    .clk_out(clk_lhz)  
);  
  
clk_wiz_0 u_pll  
(  
    .clk_out1(clk_counter),  
    .resetn(rst_n),  
    .locked(lock_counter),  
    .clk_in1(clk));  
  
endmodule
```

2.2 Clock Divider

The `clk_divider` module divides the 8 MHz clock down using a `DIVISOR` parameter. On hardware, `DIVISOR = 4,000,000` gives roughly 1 Hz. For simulation only, a smaller divisor was used instead, giving an effective 1 kHz clock so the design could be verified quickly.

```
23 module clk_divider(  
24     input clk,  
25     input rst_n,  
26     output reg clk_out  
27 );  
28  
29 //parameter DIVISOR = 4; //divisor used in simulation  
30 parameter DIVISOR = 4000000;  
31  
32 reg [22:0] count;  
33  
34 always @(posedge clk or negedge rst_n) begin  
35     if (!rst_n) begin  
36         count <= 0;  
37         clk_out <= 1'b0;  
38     end else begin  
39         if (count == DIVISOR - 1) begin  
40             count <= 0;  
41             clk_out <= ~clk_out;  
42         end else begin  
43             count <= count + 1'b1;  
44         end  
45     end  
46 end  
47  
48 endmodule  
49
```

2.3 Counter

The counter module is a 3-bit up-counter with active-low reset. It increments on every rising edge of its clock input.

```
module counter(  
    input clk,  
    input rst_n,  
    output reg [2:0] counter  
);  
  
always @(posedge clk or negedge rst_n) begin  
    if (!rst_n) begin  
        counter <= 3'b000;  
    end else begin  
        counter <= counter + 1'b1;  
    end  
end  
  
endmodule
```

2.4 Testbench

The testbench `tb_top` instantiates the top module, generates a 100 MHz clock, and applies a reset pulse. The counter, clock, and reset signals are monitored during simulation.

```
`timescale 1ns/1ps

module tb_top;

    reg clk;
    reg rst_n;
    wire [2:0] counter;

    // DUT
    top dut (
        .clk(clk),
        .rst_n(rst_n),
        .counter(counter)
    );

    // 100 MHz clock (10 ns period)
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // Reset sequence
    initial begin
        rst_n = 1'b0;

        #100;
        rst_n = 1'b1;

        #1000000;
        $stop;
    end

    // Monitor signals
    initial begin
        $monitor("Time=%0t rst_n=%b counter=%d", $time, rst_n, counter);
    end

endmodule
```

2.5 Timing Constraints

The input clock is constrained to 10 ns (100 MHz), and a generated clock is defined for the PLL output. False paths are set between the lock/reset signal and the counter, since these signals change far too slowly to need timing analysis.

```
create_clock -name clk -period 10 [get_ports clk]
create_generated_clock -name counter_clk \
    -source clk \
    -divide_by [expr 100/8] \
    u_pll/clk_out1

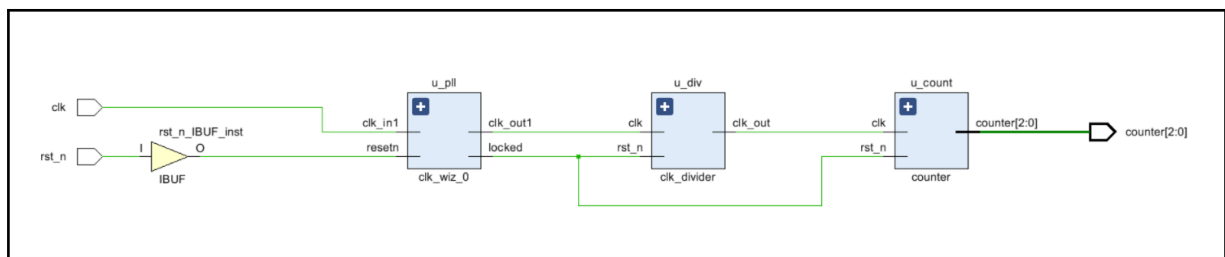
set_false_path -from lock_counter -to counter_reg[0]/CLR
set_false_path -from lock_counter -to counter_reg[1]/CLR
set_false_path -from lock_counter -to counter_reg[2]/CLR

set_false_path -from counter_reg[0]/C -to counter[0]
set_false_path -from counter_reg[1]/C -to counter[1]
set_false_path -from counter_reg[2]/C -to counter[2]
```

3 Design and Results

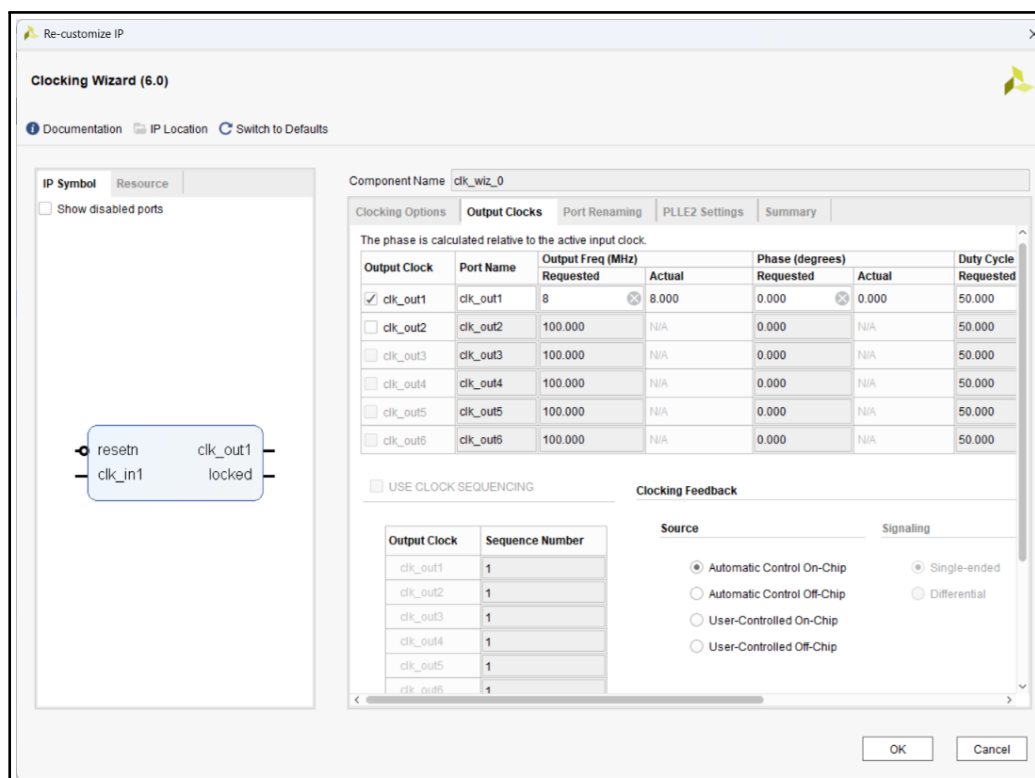
3.1 Elaborated Design Schematic

The elaborated schematic shows the PLL (u_pll), divider (u_div), and counter (u_count) as connected blocks.

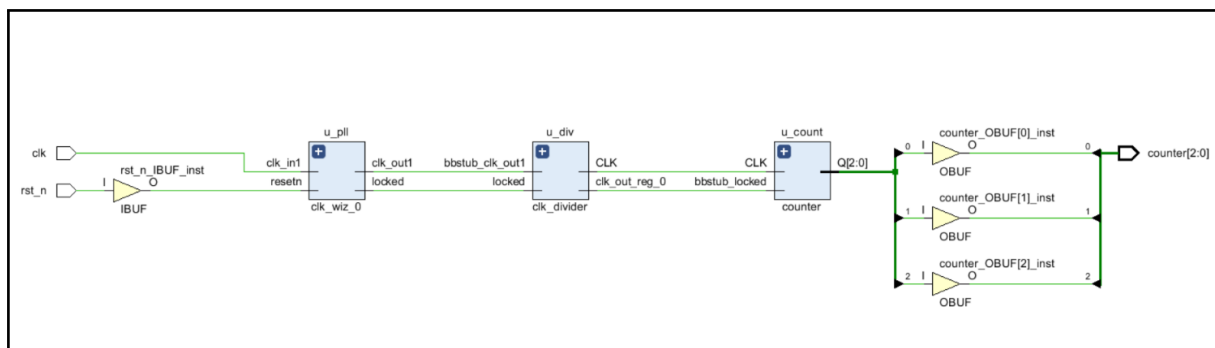


3.2 Clocking Wizard IP Configuration

The Clocking Wizard was set to output 8 MHz on clk_out1. Its locked signal is used as the reset for the rest of the design, so counting only starts once the PLL is stable.

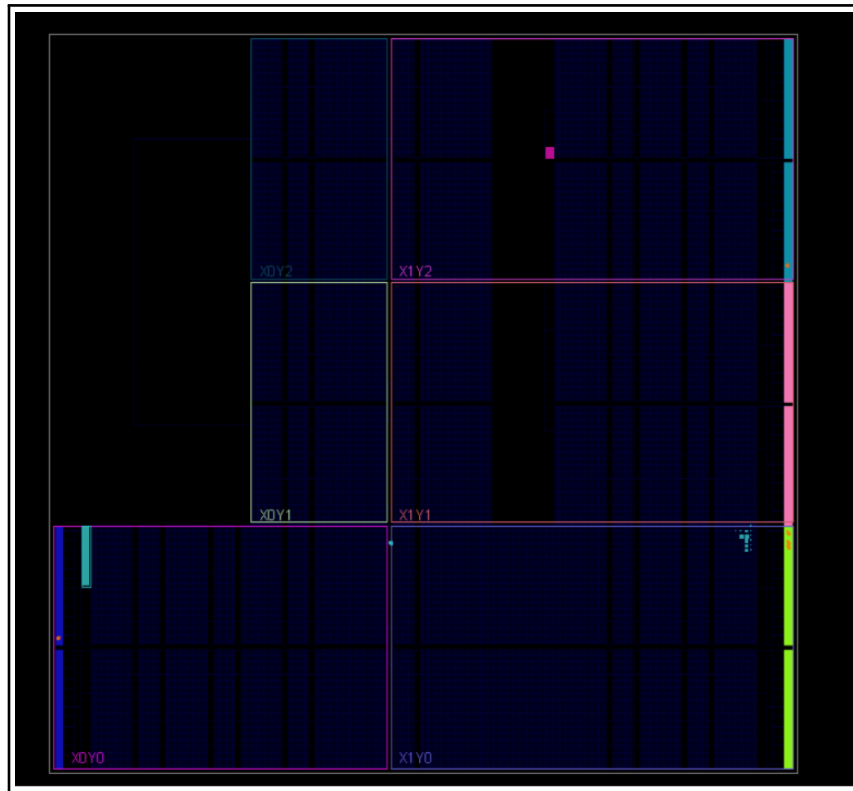


3.3 Synthesized Design



3.4 Implemented Design (Device Floorplan)

The design was placed on the xc7z020c1g484-2 floorplan as shown below.



3.5 Simulation Waveform

The waveform below confirms correct behavior: the PLL clock (`clk_counter`) toggles quickly, the divided clock (`clk_1hz`) toggles much more slowly, and the counter increments on each rising edge of the divided clock, wrapping from 7 back to 0.

